TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:

ENERO - AGOSTO 2026

CARRERA:

INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

PATRONES DE DISEÑO

UNIDAD A EVALUAR:

II

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

VELAZQUEZ MORALES LUCERO - 22210362

NOMBRE DE MAESTRO(A):

GUERRERO LUIS MARIBEL

Analizar el código proporcionado.

Program.cs

```
singleton | Program | Main(string[] args)

using singleton;
using System;

0 referencias
internal class Program
{
    0 referencias
    static void Main(string[] args)
    {
        Central_911 Llamada1 = Central_911.Obtener_Instancia();
        Central_911 Llamada2 = Central_911.Obtener_Instancia();

        Operador op1 = new Operador(1, "Laura");
        Operador op2 = new Operador(2, "Carlos");

        Llamada1.ConectarLlamada(op1, "Incendio");
        Llamada2.ConectarLlamada(op2, "Violeta");
        Llamada1.ConectarLlamada(op1, "Accidente");
        Llamada2.ConectarLlamada(op2, "Intento de suicidio");

        Console.WriteLine("\nReferenceEquals: " + ReferenceEquals(Llamada1, Llamada2));

        Console.ReadKey();
    }
}

119 % | No se encontraron problemas. | Línea: 23, Carácter: 2 | SPC | CRLF | UTF-8 with BOM
```

Central_911.cs

```
singleton | singleton.Central_911 | Obtener_Instancia()

using System;

namespace singleton
{
    8 referencias
    internal  class Central_911
    {
        private static Central_911 _instance;
        private static readonly object _lock = new object();

        1 referencia
        public string Central { get; private set; }

        1 referencia
        private Central_911()
        {
            Central = "Central 911";
        }
        2 referencias
        public static Central_911 Obtener_Instancia()
        {
            if (_instance == null)
            {
                lock (_lock)
                {
                    if (_instance == null)
                    {
                        _instance = new Central_911();
                    }

                }
            }
            return _instance;

        }
        4 referencias
        public void ConectarLlamada(Operador operador, string tipoEmergencia)
        {
            Console.WriteLine("\nLlamada conectada con el operador " + operador.Nombre);
            operador.AtiendeEmergencia(tipoEmergencia);
        }
    }

}

81 % | 0 | 1 | Línea: 40, Carácter: 1 | SPC | CRLF | UTF-8 with BOM
```

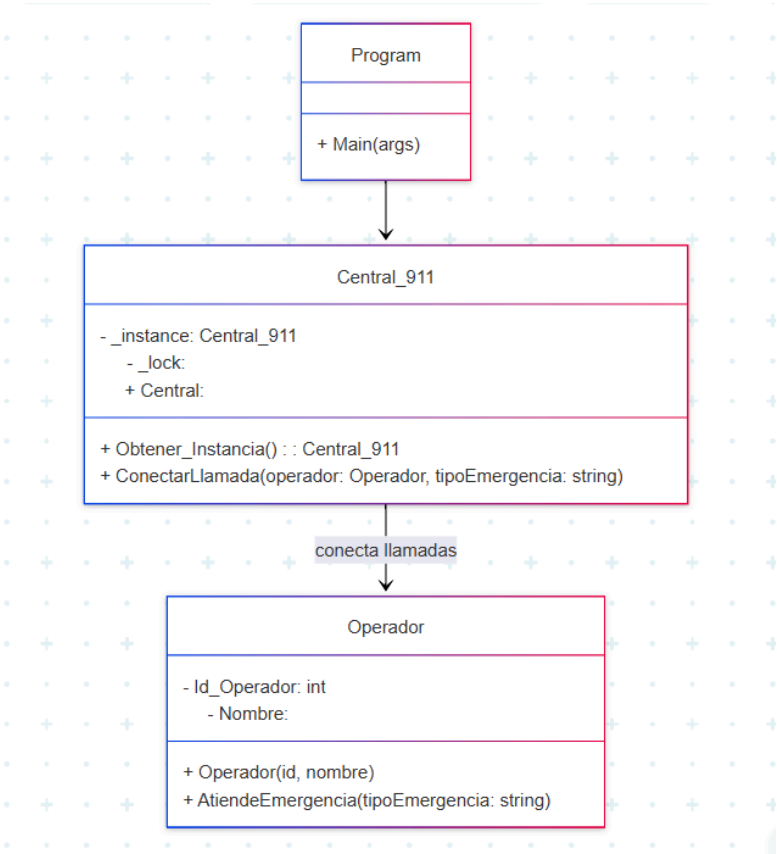
Operador.cs

```
singleton | singleton.Operador | Operador(int id, string nombre)

using System;
using System.Collections.Generic;
using System.Text;

namespace singleton
{
    6 referencias
    internal class Operador
    {
        1 referencia
        public int Id_Operador {  get; set; }
        3 referencias
        public string Nombre { get; set; }
        2 referencias
        public Operador(int id, string nombre)
        {
            Id_Operador = id;
            Nombre = nombre;
        }

        1 referencia
        public void AtiendeEmergencia(string tipoEmergencia)
        {
            Console.WriteLine($"Operador {Nombre} atendiendo emergencia de tipo: {tipoEmergencia}");

            switch (tipoEmergencia)
            {
                case "Intento de suicidio":
                    Console.WriteLine("Enviando unidades de apoyo y rescate");
                    break;
                case "Incendio":
                    Console.WriteLine("Enviando bomberos.");
                    break;
                case "Accidente":
                    Console.WriteLine("Enviando paramedicos y oficiales.");
                    break;
                case "Violeta":
                    Console.WriteLine("Enviando una patrulla.");
                    break;
                default :
                    Console.WriteLine("Tipo de emergencia no reconocido.");
                    break;
            }
        }
    }
}

74 %   No se encontraron problemas.   Línea: 42, Carácter: 1   SPC   CRLF   UTF-8 with BOM
```

Resultado

```
Consola de depuración de Mi

Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Violeta
Enviando una patrulla.

Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales.

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate

ReferenceEquals: True
```

Diagrama UML



Agregar:

- **Más operadores**

```
singleton                    Program                    Main(string[] args)
1   using singleton;
2   using System;
3
    0 referencias
4   internal class Program
5   {
        0 referencias
6       static void Main(string[] args)
7       {
8           Central_911 Llamada1 = Central_911.Obtener_Instancia();
9           Central_911 Llamada2 = Central_911.Obtener_Instancia();
10
11          Operador op1 = new Operador(1, "Laura");
12          Operador op2 = new Operador(2, "Carlos");
13          Operador op3 = new Operador(3, "Lucero");
14          Operador op4 = new Operador(4, "sofia");
15          Operador op5 = new Operador(5, "Santiago");
16          Operador op6 = new Operador(6, "Jesus");
17          Operador op7 = new Operador(7, "Lizari");
18          Operador op8 = new Operador(8, "Deyver");
19          Operador op9 = new Operador(9, "katy");
20          Operador op10 = new Operador(10, "Valentina");
21
22
23          Llamada1.ConectarLlamada(op1, "Incendio");
24          Llamada2.ConectarLlamada(op2, "Violeta");
25          Llamada1.ConectarLlamada(op1, "Accidente");
26          Llamada2.ConectarLlamada(op2, "Intento de suicidio");
27
28          Console.WriteLine("\nReferenceEquals: " + ReferenceEquals(Llamada1, Llamada2));
29
30          Console.ReadKey();
31      }
32  }
98 %   No se encontraron problemas.   Línea: 32, Carácter: 2   SPC   CRLF   UTF-8 with BOM
```

- **Agregar más llamadas**

```
singleton                    Program                    Main(string[] args)
    0 referencias
4   internal class Program
5   {
        0 referencias
6       static void Main(string[] args)
7       {
8           Central_911 Llamada1 = Central_911.Obtener_Instancia();
9           Central_911 Llamada2 = Central_911.Obtener_Instancia();
10
11          Operador op1 = new Operador(1, "Laura");
12          Operador op2 = new Operador(2, "Carlos");
13          Operador op3 = new Operador(3, "Lucero");
14          Operador op4 = new Operador(4, "sofia");
15          Operador op5 = new Operador(5, "Santiago");
16          Operador op6 = new Operador(6, "Jesus");
17          Operador op7 = new Operador(7, "Lizari");
18          Operador op8 = new Operador(8, "Deyver");
19          Operador op9 = new Operador(9, "katy");
20          Operador op10 = new Operador(10, "Valentina");
21
22
23          Llamada1.ConectarLlamada(op1, "Incendio");
24          Llamada2.ConectarLlamada(op2, "Violeta");
25          Llamada1.ConectarLlamada(op3, "Accidente");
26          Llamada2.ConectarLlamada(op4, "Intento de suicidio");
27          Llamada1.ConectarLlamada(op5, "Incendio");
28          Llamada2.ConectarLlamada(op6, "Accidente");
29          Llamada1.ConectarLlamada(op7, "Violeta");
30          Llamada2.ConectarLlamada(op8, "Incendio");
31          Llamada1.ConectarLlamada(op9, "Intento de suicidio");
32          Llamada2.ConectarLlamada(op10, "Accidente");
33
34          Console.WriteLine("\nReferenceEquals: " + ReferenceEquals(Llamada1, Llamada2));
35
36          Console.ReadKey();
37      }
38  }
98 %   No se encontraron problemas.   Línea: 22, Carácter: 1   SPC   CRLF   UTF-8 with BOM
```

- **Verificar que es una única instancia**

Se utiliza ReferenceEquals(Llamada1, Llamada2) para comprobar si ambas variables apuntan a la misma instancia de la clase Central_911. Si el resultado es True, significa que ambas variables usan el mismo objeto en memoria.

```
            Console.WriteLine("\n¿Es la misma instancia?");
            Console.WriteLine(ReferenceEquals(Llamada1, Llamada2));

            Console.ReadKey();
        }
    }
```

Resultado

```
C:\Users\lucer\source\repos\s

Operador Laura atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Violeta
Enviando una patrulla.

Llamada conectada con el operador Lucero
Operador Lucero atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales.

Llamada conectada con el operador sofia
Operador sofia atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate

Llamada conectada con el operador Santiago
Operador Santiago atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador Jesus
Operador Jesus atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales.

Llamada conectada con el operador Lizari
Operador Lizari atendiendo emergencia de tipo: Violeta
Enviando una patrulla.

Llamada conectada con el operador Deyver
Operador Deyver atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador katy
Operador katy atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate

Llamada conectada con el operador Valentina
Operador Valentina atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales.

¿Es la misma instancia?
True
```

Identificar y explicar:

1.-el atributo estático de la instancia

El atributo private static Central_911 instancia se utiliza para almacenar la única instancia de la clase. Al ser estático, pertenece a la clase y no a los objetos, lo que permite que todos los accesos a la clase utilicen la misma instancia.

```
private static Central_911 _instance;
```

2.-el constructor

El constructor private Central_911() es privado para evitar que se creen múltiples objetos de la clase. Esto permite que solo exista una única instancia de Central_911, cumpliendo con el patrón Singleton.

```
1 referencia
private Central_911()
{
    Central = "Central 911";
}
2 referencias
```

Atributo estático

- Guarda la única instancia de la clase.

Constructor privado

- Evita que se creen más objetos.

Responder:

¿Por qué no se pueden crear múltiples objetos?

No se pueden crear múltiples objetos porque el constructor de la clase es privado. Esto evita que otras clases creen instancias usando new. En su lugar, se utiliza el método Obtener_Instancia(), que devuelve siempre la misma instancia de la clase, asegurando que solo exista un objeto.

Se utilizó un ciclo for y la clase Random para seleccionar operadores y tipos de emergencia de forma aleatoria, simulando múltiples llamadas al sistema de emergencia.

singleton | Program

```
using singleton;
using System;

0 referencias
internal class Program
{
    0 referencias
    static void Main(string[] args)
    {
        Central_911 Llamada1 = Central_911.Obtener_Instancia();
        Central_911 Llamada2 = Central_911.Obtener_Instancia();

        Operador op1 = new Operador(1, "Laura");
        Operador op2 = new Operador(2, "Carlos");
        Operador op3 = new Operador(3, "Lucero");
        Operador op4 = new Operador(4, "sofia");
        Operador op5 = new Operador(5, "Santiago");
        Operador op6 = new Operador(6, "Jesus");
        Operador op7 = new Operador(7, "Lizari");
        Operador op8 = new Operador(8, "Deyver");
        Operador op9 = new Operador(9, "katy");
        Operador op10 = new Operador(10, "Valentina");


        Operador[] operadores = { op1, op2, op3, op4, op5, op6, op7, op8, op9, op10 };

        //tipos de emergencia
        string[] emergencias =
        {
            "Incendio",
            "Accidente",
            "Violencia",
            "Intento de suicidio",
            "Emergencia médica"
        };


        Random rnd = new Random();


        for (int i = 0; i < 10; i++)
        {
            Operador operadorRandom = operadores[rnd.Next(operadores.Length)];
            string emergenciaRandom = emergencias[rnd.Next(emergencias.Length)];

            Llamada1.ConectarLlamada(operadorRandom, emergenciaRandom);
        }

        Console.WriteLine("\n¿Es la misma instancia?");
        Console.WriteLine(ReferenceEquals(Llamada1, Llamada2));

        Console.ReadKey();
    }
}
```

67 % No se encontraron problemas.

singleton | singleton.Operador

```
1 referencia
public int Id_Operador {  get; set; }
3 referencias
public string Nombre { get; set; }
10 referencias
public Operador(int id, string nombre)
{
    Id_Operador = id;
    Nombre = nombre;
}

1 referencia
public void AtiendeEmergencia(string tipoEmergencia)
{
    Console.WriteLine($"Operador {Nombre} atendiendo emergencia de tipo: {tipoEmergencia}");

    switch (tipoEmergencia)
    {
        case "Intento de suicidio":
            Console.WriteLine("Enviando unidades de apoyo y rescate");
            break;
        case "Incendio":
            Console.WriteLine("Enviando bomberos.");
            break;
        case "Accidente":
            Console.WriteLine("Enviando paramedicos y oficiales.");
            break;
        case "Violeta":
            Console.WriteLine("Enviando una patrulla.");
            break;
        case "Emergencia médica":
            Console.WriteLine("Enviando ambulancia.");
            break;
        case "Violencia":
            Console.WriteLine("Enviando patrulla y apoyo.");
            break;
        default :
            Console.WriteLine("Tipo de emergencia no reconocido.");
            break;
    }
}
}
}
```

```
C:\Users\lucer\source\repos\s

Operador Laura atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Violencia
Enviando patrulla y apoyo.

Llamada conectada con el operador Valentina
Operador Valentina atendiendo emergencia de tipo: Violencia
Enviando patrulla y apoyo.

Llamada conectada con el operador Valentina
Operador Valentina atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador katy
Operador katy atendiendo emergencia de tipo: Violencia
Enviando patrulla y apoyo.

Llamada conectada con el operador Santiago
Operador Santiago atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales.

Llamada conectada con el operador Santiago
Operador Santiago atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate

Llamada conectada con el operador Lizari
Operador Lizari atendiendo emergencia de tipo: Violencia
Enviando patrulla y apoyo.

Llamada conectada con el operador katy
Operador katy atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales.

Llamada conectada con el operador Deyver
Operador Deyver atendiendo emergencia de tipo: Violencia
Enviando patrulla y apoyo.

¿Es la misma instancia?
True
```

Conclusión

Este patrón Singleton permite que solo exista una única instancia de una clase, en este caso la clase central_911, lo cual se comprobó utilizando ReferenceEquals, también se crearon varios operadores y diferentes emergencias para simular llamadas al sistema